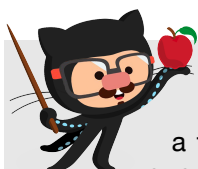


DevWeb

Capítulo 24

Formulários

Estamos na reta final do nosso aprendizado básico com HTML5 e agora chegou a hora de aprender a criar formulários, pois eles são muito usados em sites. Toda tela de login, toda página de cadastro, todo campo de busca vai precisar obrigatoriamente de um formulário HTML para começar. E o que vamos aprender aqui é mais do que apenas criar formulários que funcionam visualmente. Acompanhe comigo em detalhes a partir de agora!



Você tem todo o direito de usar esse material para seu próprio aprendizado. Professores também podem ter acesso a todo o conteúdo e usá-los com seus alunos. Porém todos o que usarem esse material - seja para qual for a finalidade - deverão manter a referência ao material original, criado pelo **Prof. Gustavo Guanabara** e disponível no endereço do seu repositório público <https://github.com/gustavoguanabara/>. Este material não poderá ser utilizado em nenhuma hipótese para ser replicada - integral ou parcialmente - por autores/editoras para criar livros ou apostilas, com finalidade de obter ganho financeiro com ele.

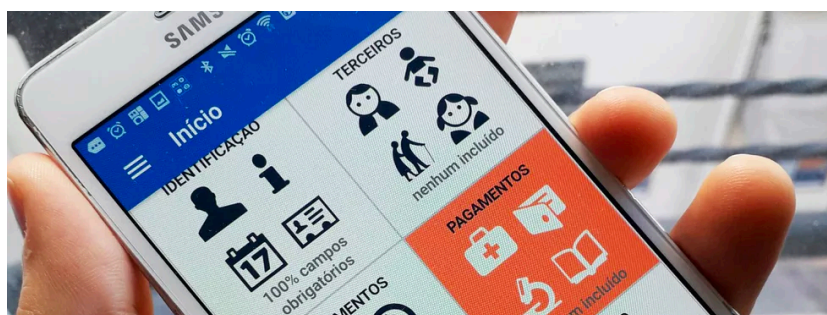


Formulários são essenciais

Vamos começar com uma curiosidade (onde também vou vergonhosamente reconhecer que estou ficando velho) e lembrar algo que talvez nem faça parte da sua memória. Provavelmente você ou seus familiares fazem a **Declaração de Imposto de Renda** no início de cada ano usando o computador ou celular. Pois saiba que antigamente, antes dos computadores pessoais se tornarem algo popular, essa declaração era feita em papel, preenchendo um formulário de papel gigantesco que era entregue fisicamente.



De lá pra cá, as coisas evoluíram e hoje é possível usar até mesmo o seu celular para se manter em dia com a Receita Federal. O formulário gigantesco ainda está lá, mas pode ser preenchido de forma digital e muito mais organizada e sem dores de cabeça (além daquelas naturais, no momento de pagar os impostos).



Eu já deveria ter ensinado formulários desde as primeiras aulas de HTML5, mas uma avalanche de *tags* e *propriedades* acabou me impedindo de fazer isso mais cedo, mas você pode ter certeza que ter deixado esse conceito por último (até depois dos *iframes*) não é um desmerecimento ou algum tipo de "castigo". Na verdade, tudo aquilo que você aprendeu até hoje te trouxe até aqui e os formulários são a cerejeira desse bolo.

Atualmente, encontramos formulários em várias ocasiões, enquanto navegamos em sites:

- Toda tela de login, onde digitamos usuário e senha (e às vezes outras coisas)
- Aquelas janelas onde informamos nosso e-mail para cadastrar em *newsletter* e muitas vezes nos arrependemos, pois começamos a receber muito SPAM.
- Toda página de cadastro, onde informamos dados pessoais como nome, e-mail, endereço, etc.
- Aqueles campos de busca que encontramos geralmente no cabeçalho do site e usamos para encontrar conteúdos perdidos em meio de tantos artigos.

Um formulário simples não pode ser tão simples

Quando aprendemos a criar um formulário, é comum encontrar exemplos como no código a seguir:

```
<form>
  Nome:
  <input type="text" name="fname">
  <input type="submit" value="Enviar">
</form>
```

A primeira e a última linhas vão delimitar o formulário. Tudo aquilo que estiver dentro desse par de *tags* será considerado como conteúdo dos campos. No exemplo acima, temos apenas um campo que vai aceitar um texto relativo ao nome da pessoa e um botão de submissão, que vai permitir enviar o formulário. O resultado do código acima será:

Nome:

E aí você pode pensar: *"-Pronto! Está funcionando!"*. Infelizmente não! Precisamos fazer algumas configurações adicionais para que esse formulário funcione de maneira mais semântica e correta.

- Em primeiro lugar, vamos adicionar dois atributos importantes à tag `<form>`: o `method` e o `action`. O primeiro vai indicar o método de envio dos dados e o segundo vai dizer para onde os dados serão enviados.
- Não existe nenhuma ligação semântica entre a caixa de texto e seu identificador. Para isso, vamos adicionar um `<label>` para que possa ser feito o relacionamento entre o campo e seu identificador.
- Para que o `<label>` possa funcionar corretamente, a caixa de texto deverá ter um `id`.

Depois de fazer as três alterações indicadas acima, teremos o novo código:

```
<form method="GET" action="cadastro.php">
  <label for="camponome">Nome:</label>
  <input type="text" name="fname" id="camponome">
  <input type="submit" value="Enviar">
</form>
```

E o resultado visual será exatamente o mesmo:

Nome:

E nesse momento você também deve estar imaginando: *"-Mas antes estava igual, pra que adicionar essas coisas?"*. A resposta é simples.

O formulário está **visualmente** igual, mas agora ele está mais **semanticamente** correto e o seu navegador vai conseguir relacionar os campos e enviar os dados da maneira certa para o lugar certo. Na primeira versão, mais simples, o seu navegador seria capaz de captar um dado, mas não saberia para onde enviar.



E NADA DE PREGUIÇA! Todas as vezes em que você se pegar pensando "*mas aí eu estou escrevendo muito...*", pense que você está estudando para exercer uma profissão! E um profissional que desenvolve sites realmente deve estar disposto a - eventualmente - ter bastante trabalho. Faça as coisas do jeito certo, não economize no código e seus projetos serão mais relevantes. Fica a dica!

E quais métodos posso usar?

No exemplo anterior, configuramos o `method` do formulário para o valor `GET`, mas eu não expliquei como funciona esse método e nem quais são as opções disponíveis para esse atributo. Basicamente, existem dois métodos para enviar os dados de um formulário: `GET` e `POST`.

Para exemplificar o método `GET`, acesse agora o site do **Google** e faça uma busca por *Curso HTML5*.

Depois de enviar o formulário (sim, a página principal do Google é um formulário), preste bastante atenção à barra de endereços do navegador e veja a `URL` que você está acessando:

Note que aquilo que você digitou no formulário apareceu na `URL` que está sendo apresentada no seu navegador. Isso é uma característica presente nos formulários com método `GET`, pois ele condensa todos os dados digitados pelo visitante e envia como parte do endereço acessado.

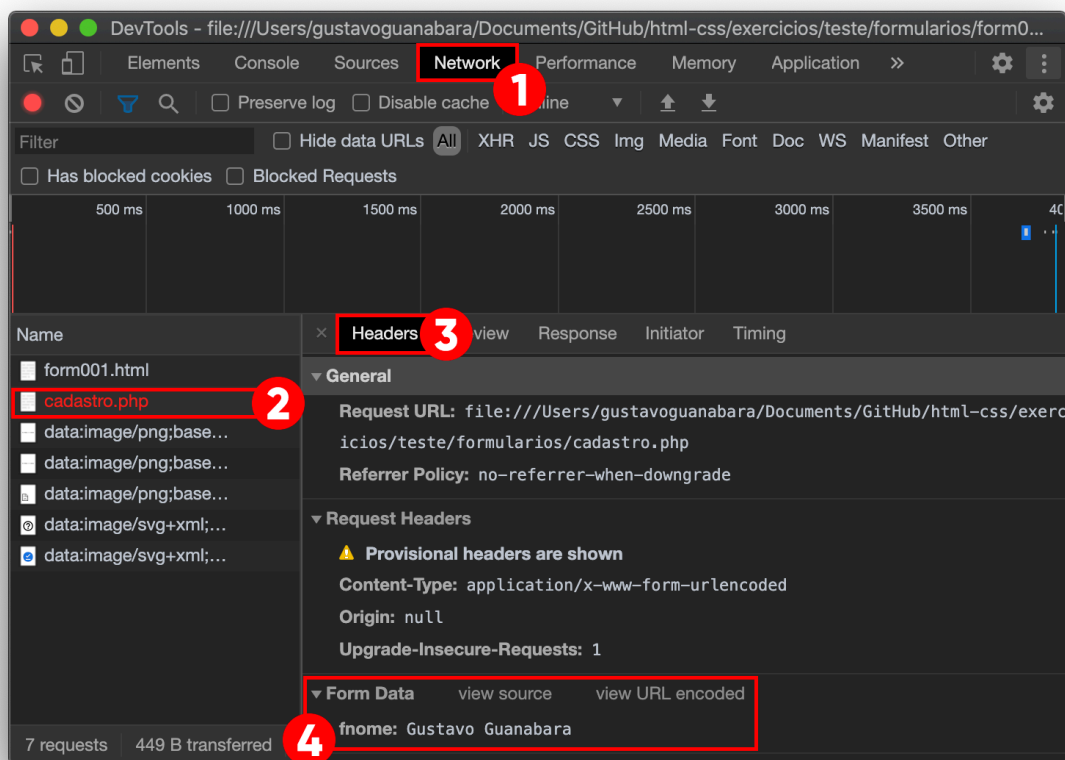
Já o método `POST` vai pegar os dados digitados e enviar diretamente no corpo da requisição `HTTP` que será feita quando seu visitante clicar para enviar o formulário. Sendo assim, os dados não vão aparecer na `URL`.

Mas quais são as diferenças entre GET e POST?

Já vi muita gente dizendo que o método POST tinha mais “segurança” que o método GET só porque os dados não aparecem na *URL*. Isso não é bem verdade. Basta mudar o método do seu formulário para POST, abrir o seu formulário no **Google Chrome** e chamar o **DevTools**. Em seguida, digite seu nome no formulário e pressione Enviar.

Nome:

Agora vá até a aba **Network** e escolha a chamada para o arquivo configurado no action do formulário, no nosso caso, cadastro.php. Clique sobre a aba **Headers** e em seguida procure a seção **Form Data**. Pronto! Lá estão os dados.



E já que esclarecemos que não é uma questão de segurança, vamos analisar em que momentos vamos usar cada um dos métodos.

Em primeiro lugar, podemos usar GET em um formulário contanto que:

- Os dados a serem enviados não sejam sensíveis, como endereços, números de documentos, senhas, etc.
- Tivermos formulários simples e que possam ser compartilhados facilmente através de uma URL. Por exemplo, o Google usa GET no seu formulário, pois você pode enviar um link para qualquer pessoa, já com a ação da busca embutida, como em <http://google.com.br/search?q=Gustavo+Guanabara>

- Os dados a serem enviados nunca ultrapassem 3000 bytes.
- Não contenha envio de arquivos como imagens (i.e. fotos de perfil), arquivos PDF, etc.

Já o método POST é mais recomendado para casos em que:

- Não queremos que os dados apareçam explicitamente na URL do navegador.
- Capturamos dados como senhas, documentos e outro tipo de dado sensível.
- Precisamos enviar muitos dados (acima de 3000 bytes ~ sem limites).
- Precisamos enviar imagens ou outros tipos de arquivos.



A DISCUSSÃO É AINDA MAIS PROFUNDA, porém não vou me aprofundar aqui nos assuntos que envolvem as utilizações de métodos de envio de formulários, já que o curso aqui é para conceitos introdutórios. Mas se quiser mais detalhes, consulte esse tópico (em inglês):

Stack Overflow: <https://stackoverflow.com/questions/46585/>

Outra discussão que é bastante levantada nas comunidades é em relação à velocidade ou performance de cada método. De forma resumida, um formulário que usa método GET é ligeiramente mais rápido que um formulário que usa o POST. Porém, não use isso como critério para decidir qual método aplicar ao seu formulário. Foque-se primordialmente nos pontos levantados logo acima e sua escolha será muito mais sábia.

Usar name ou id: qual é o mais útil e recomendado?

Outra dúvida bem comum entre os meus alunos é em relação ao uso dos atributos name e id em elementos de formulários. Pode parecer que os dois servem para identificar os elementos e que o uso de ambos é algo desnecessário, mas não é. Vamos dar uma olhada na linha do exercício que estamos criando:

```
<input type="text" name="fname" id="camponome">
```

De forma objetiva, **use sempre os dois** em todos os seus elementos, pois cada um terá sua utilidade específica.

O name será bem útil durante o envio dos dados do formulário, independente do método escolhido para enviá-los. É ele que aparece na *URL* (ao usar GET) e no corpo do *HTTP request* (ao usar POST). É também através do name escolhido que vamos nos referenciar quando for a hora de programar o arquivo em *PHP* que vai receber esses dados.

Já o id será muito útil para fazer a ligação com o <label> (sempre use *labels*, por favor) e também será referenciado nos casos em que quisermos tratar esses dados

usando *JavaScript* para realizar qualquer ação, como por exemplo, fazer validação local dos campos antes de enviá-los.

Sendo assim, já disse mas repito: use sempre os dois atributos. Todo elemento de formulário que contenha dados deverá ter `name` e `id`. Nada de preguiça!

As múltiplas faces do `<input>`

A tag `<input>` é uma das mais versáteis da linguagem HTML5 (linguagem de marcação, não de programação, nunca se esqueça), e digo isso porque é possível criar muitos componentes de formulário usando apenas essa *tag*. Veja a seguir os seguintes exemplos:

Foto do perfil: Nenhum arquivo selecionado

Idade:

Sexo

☒ Masculino

☐ Feminino

Telefone:

E-mail:

Senha:

Mês de aniversário:

Dia da compra:

Hora da compra:

Cor:

Nível de satisfação:

☒ Aceito os [termos de uso](#)

Todos os elementos acima, por mais que possam parecer visualmente diferentes, foram criados essencialmente usando `<label>` e `<input>` na sua construção (exceto no campo sexo, onde foi preciso criar um `<fieldset>`). O trecho completo do código usado para criar o formulário acima será apresentado a seguir, e analisaremos suas principais configurações logo depois, sempre referenciando o número da linha para melhor organização e entendimento.

Mas antes de olhar a próxima página, um aviso: não se assuste! Vai ficar tudo bem!

```

1      <form action="cadastro.php" method="get">
2          <p><label for="ifoto">Foto do perfil:</label>
3          <input type="file" name="foto" id="ifoto" accept="image/*"></p>
4          <p><label for="iidade">Idade:</label>
5          <input type="number" name="idade" id="iidade" min="1" max="120"
6          value="20"></p>
7          <p><fieldset style="width: 100px;">
8              <legend>Sexo</legend>
9              <input type="radio" name="sex" id="isexmasc" checked>
10             <label for="isexmasc">Masculino</label>
11             <input type="radio" name="sex" id="isexfem">
12             <label for="isexfem">Feminino</label>
13         </fieldset></p>
14         <p><label for="itel">Telefone:</label>
15         <input type="tel" name="tel" id="itel" pattern="\\([0-9]{2}\\)([\\s])"
16         [0-9]{4,6}-[0-9]{4}" placeholder="(XX) XXXX-XXXX"></p>
17         <p><label for="imail">E-mail:</label>
18         <input type="email" name="mail" id="imail" size="30"
19         maxlength="50"></p>
20         <p><label for="isenha">Senha</label>
21         <input type="password" name="senha" id="isenha" minlength="3"
22         maxlength="8" size="8"></p>
23         <p><label for="imes">Mês de aniversário:</label>
24         <input type="month" name="mes" id="imes"></p>
25         <p><label for="idata">Dia da compra:</label>
26         <input type="date" name="data" id="idata"></p>
27         <p><label for="ihora">Hora da compra:</label>
28         <input type="time" name="hora" id="ihora"></p>
29         <p><label for="icor">Cor:</label>
30         <input type="color" name="cor" id="icor" value="#000000"></p>
31         <p><label for="isat">Nível de satisfação:</label>
32         <input type="range" name="sat" id="isat" min="0" max="4"
33         value="2"></p>
34         <p><input type="checkbox" name="aceito" id="iaceito">
35         <label for="iaceito">Aceito os <a href="termos.html">termos de
36         uso</a></label></p>
37         <p><input type="submit" value="Enviar">
38         <input type="reset" value="Limpar"></p>
39     </form>

```

O formulário se inicia na **linha 1** e termina na **linha 33**. Até aí, tudo bem! Note que cada um dos campos do formulário foi devidamente limitado dentro de um parágrafo. Use esses parágrafos para servir de referência e aprender a usar cada um dos elementos.

Nas **linhas 2 e 3**, criamos um campo do tipo file, que permite o visitante fazer o upload de arquivos em qualquer formato. No exemplo dado, usamos o atributo accept

para limitar o upload a arquivos de imagem, já que queremos o envio a penas de fotos de perfil.



LISTA COMPLETA DE FORMATOS: Você pode consultar o site da IANA para uma listagem com todos os formatos suportados.

Media Types: <http://www.iana.org/assignments/media-types/media-types.xhtml>

Nas **linhas 4 e 5**, usamos o tipo `number` para criar uma caixa que aceita apenas números. Inclusive, usamos os atributos `min` e `max` para definir os limites aceitos e o atributo `value` para sugerir um valor inicial, que pode ser alterado para qualquer outro valor dentro do limite durante o preenchimento do formulário.

As **linhas 6 até 12** definem um grupo de campos na forma de um `<fieldset>`. Por padrão, é exibida uma linha em volta do controle (pode ser personalizada via CSS) e permite usar a tag `<legend>` para descrever qual será o texto que identifica o grupo.

Note que na **linha 8** e na **linha 10**, os botões de rádio (aqueles redondinhos que podem ser selecionados individualmente) possuem o mesmo `name`. Isso é necessário para indicar que os dois fazem parte do mesmo grupo a ser selecionado. Apenas um dele pode ser marcado e o primeiro já vai aparecer pressionado, pois usamos `checked` nele. Seus `id` são diferentes, para que possamos usar os *labels* corretamente.



CUIDADO! Quando estiver usando botões de radio, todos aqueles que fazem parte do mesmo grupo, devem ter o mesmo nome! Faça uma experiência, coloque nomes diferentes e veja o resultado ao tentar marcá-los!

Nas **linhas 13 e 14**, usamos um campo do tipo `tel` para ler um número de telefone no atributo `pattern` configuramos uma expressão regular para definir qual será o formato aceito para o valor. Já o atributo `placeholder` vai configurar uma “dica” para que seu visitante saiba exatamente qual é o formato que vai usar para informar seu telefone.

- `\([0-9]{2}\)` vai indicar que é necessário colocar dois dígitos de 0 a 9 entre parênteses.
- `([\s])` indica que logo em seguida, teremos um espaço, logo após o fechamento dos parênteses.
- `[0-9]{4,6}` - configura que podem existir entre 4 e 6 dígitos numéricos entre 0 e 9 e logo depois, um traço.
- `[0-9]{4}` por fim, o número se encerra com 4 dígitos entre 0 e 9.

As **linhas 15 e 16** servem para solicitar um e-mail, e isso vai exigir que o campo tenha uma `@` para ser considerado válido. Também configuramos o tamanho máximo com `maxlength` e nenhum endereço digitado poderá conter mais de 50 caracteres. Também configuramos o tamanho físico da caixa para exibir 30 caracteres por vez ao

usar o `size`. Esses últimos dois atributos também podem ser usados em vários outros tipos de campo.

Isso se comprova, pois nas **linhas 17 e 18** criamos um campo para solicitar a senha e também usamos eles. Além disso, adicionamos o atributo `minlength` que em conjunto com o `maxlength`, vai indicar que nossa senha vai ter entre 3 e 8 caracteres.

No trecho de código que engloba as **linhas 19 até 24**, usamos controles especiais para data/hora. Experimente clicar no ícone que fica no final da caixa para ver que o navegador exibe blocos que facilitam a seleção em cada caso.

Nas **linhas 25 e 26**, criamos um controle que permite a seleção de cores e também usamos o atributo `value` para definir a cor padrão, que também pode ser alterada pelo visitante.

As **linhas 27 e 28** foram configuradas para exibir um range, exibido como uma pequena barra horizontal com um controle deslizante. Nesse elemento, usamos os atributos `min` e `max` para definir os limites e o `value` para indicar qual será o valor selecionado inicialmente.

Configuramos nas **linhas 29 e 30** um único checkbox, que se diferem dos botões de radio porque podem ter vários itens selecionados dentro de um mesmo grupo. Por conta disso, siga exatamente a regra **oposta** àquela que vimos anteriormente para os radio: **nunca coloque o mesmo nome** em vários checkbox, ok? Cada um deve ter um nome e um id diferentes.



CUIDADO! Ao usar um grupo de checkbox em um mesmo formulário, cada um deve ter seu próprio nome e id. Jamais repita um mesmo nome ou id para checkboxes diferentes.

Por fim, nas **linhas 31 e 32**, criamos dois botões: um submit e um reset. O primeiro vai capturar todos os dados digitados pelo visitante nos campos do formulário e enviar para o documento indicado dentro do atributo `action` da tag `<form>` onde ele está contido. Já o segundo botão vai limpar todos os dados digitados e restaurar todos os valores padrão dos elementos (caso existam).

Meu controle de formulário não apareceu como o seu. E agora?

Provavelmente, ao criar seu formulário em casa, os componentes foram apresentados de forma um pouco diferente, mas saiba que isso é super natural, pois se tratam de plataformas diferentes (estou desenvolvendo o material no MacOS X, por exemplo) e também existe diferença de renderização ao usar outros navegadores como o **Firefox**, o **Edge**, o **Safari** e o **Opera**.



As dicas para criar uma experiência próxima do ideal é fazer testes no maior número de navegadores e usar folhas de estilo para formatar visualmente cada componente para não deixar o navegador decidir o formato deles. Outro grande problema é em relação à compatibilidade com alguns tipos de elementos da HTML5. Alguns navegadores sequer suportam campos como `color`, `range` e `time`, por exemplo. Isso se resolve com o tempo, conforme as versões atuais do *browser* aumente sua compatibilidade com as novidades constantes da HTML5 e das CSS.

E paramos por aqui? Só temos `<form>` e `<input>`?

E não é só de `input` que nossos formulários podem ser construídos. Existem outras opções de elementos para aumentar as possibilidades. Vamos ver mais algumas possibilidades:

Modelo do celular:

Especificações técnicas:

Profissão:

No formulário acima, temos uma caixa combinada, daquelas que podemos clicar e escolher um dos itens que vão aparecer em uma lista pré-definida. Logo a seguir, temos uma área de texto, onde digitamos conteúdo em vários parágrafos. Por fim, temos uma caixa de texto comum, mas que possui também uma lista incorporada. Podemos escolher um dos itens disponíveis nessa listagem ou digitar nossa própria resposta.

Mais uma vez, vamos ver como foi possível construir cada um dos elementos do formulário acima, através do trecho de código que utilizamos no documento HTML. Analise cada uma das linhas do código a seguir para poder entender como foi possível criar os elementos.

```

1      <form action="cadastro.php" method="get">
2          <p><label for="imod">Modelo do celular:</label>
3          <select name="modelo" id="imod">
4              <option value="">Escolha...</option>
5              <optgroup label="Apple">
6                  <option value="AIP1132">iPhone 11 32GB</option>
7                  <option value="AIP11P64">iPhone 11 Pro 64GB</option>
8                  <option value="AIP11PM256">iPhone 11 Pro Max 256GB</option>
9              </optgroup>
10             <optgroup label="Samsung">
11                 <option value="SGXYS10">Galaxy S10 Lite</option>
12                 <option value="SGXYS20U128">Galaxy S20 Ultra 128GB</option>
13                 <option value="SGXYN10256">Galaxy Note10 256GB</option>
14             </optgroup>
15         </select></p>
16         <p><label for="idesc">Especificações técnicas:</label><br>
17         <textarea name="desc" id="idesc" cols="40" rows="10"></textarea></p>
18         <p><label for="iprof">Profissão:</label>
19         <input type="text" name="prof" id="iprof" list="lstprof">
20         <datalist id="lstprof">
21             <optgroup label="Negócios">
22                 <option value="Administrador"></option>
23                 <option value="Contabilista"></option>
24                 <option value="Gestor de RH"></option>
25             </optgroup>
26             <optgroup label="Artes e Design">
27                 <option value="Animador"></option>
28                 <option value="Ator"></option>
29                 <option value="Fotógrafo"></option>
30             </optgroup>
31             <optgroup label="Ciências Exatas">
32                 <option value="Analista de Sistemas"></option>
33                 <option value="Estatístico"></option>
34                 <option value="Matemático"></option>
35             </optgroup>
36             <optgroup label="Ciências Humanas">
37                 <option value="Advogado"></option>
38                 <option value="Linguista"></option>
39                 <option value="Professor"></option>
40             </optgroup>
41         </datalist></p>
42         <p><input type="submit" value="Enviar"></p>
43     </form>

```

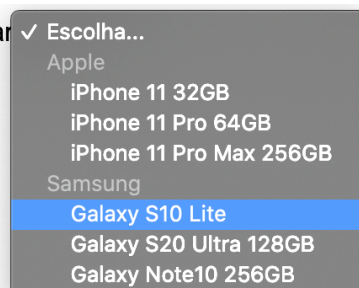
Com toda a certeza esses códigos estão ficando mais claros pra você agora. E se por acaso ainda estiver confuso, pode ser sinal de que você não está praticando o suficiente. Você vai ver que com a prática, tudo vai ficando mais simples. Vai por mim, invista um tempo para praticar.

Para começar, criamos um elemento de lista do tipo *dropdown* ao criar o código que está no intervalo que vai da **linha 2 até 15**. Tudo começa, criando a tag `<select>` e dando a ela a identificação necessária através do `id` e do `name`. Cada uma das opções deverá vir entre `<select>` e `</select>`. As opções também podem ser agrupadas em `<optgroup>`, separando-as por assunto caso seja necessário. Caso não exista essa divisão, você pode colocar todos os `<option>` diretamente dentro do `<select>`.

O resultado visual obtido com o código acima para a criação da lista está representado aí ao lado, incluindo a separação em grupos.

Cada `<option>` possui seu `value`, que é o valor que será enviado caso o seu visitante escolha essa opção. Geralmente, quando trabalhamos com listas desse tipo, enviamos pelo formulário o código de cada item, de acordo com o que temos cadastrado em um banco de dados, por exemplo.

Modelo do celular



Logo a seguir, nas **linhas 16 e 17**, temos uma área de texto que vai aceitar múltiplas linhas. Nesse caso, os parâmetros `rows` e `cols` definem respectivamente em quantas linhas e quantas colunas o conteúdo será exibido, mas saiba que o conteúdo completo pode ter muito mais que o número de linhas configurado. A capacidade de uma `<textarea>` é **ilimitada**.

Profissão:



O último elemento que criamos nesse formulário foi uma caixa de texto comum, definida diretamente nas **linhas 18 e 19**. A única diferença é a presença do atributo `list`, que vai apontar para uma `<datalist>` que está sendo criada logo abaixo, no intervalo que vai da **linha 20 até 41**.

Assim como `<select>`, uma `<datalist>` também pode conter vários `<option>` em seu interior. Visualmente, a caixa de texto vai aceitar qualquer valor personalizado, mas sempre vai sugerir os valores que definimos dentro das opções, inclusive filtrando os valores exibidos conforme vamos digitando o conteúdo da caixa de texto.

Perceba que durante a preparação desse material que você tem em mãos agora, o uso de `<optgroup>` dentro de uma `<datalist>` não está sendo suportado pelo **Google Chrome**. Faça seus testes em casa e veja se já existe compatibilidade ou não.

Gerando saídas rápidas utilizando `<output>`

Um recurso que foi adicionado à HTML5 foi o elemento `<output>`, que pode ser usado para executar tarefas simples e gerar uma saída na tela. Essa execução é feita usando

a Linguagem de Programação JavaScript e pode ser executada diretamente no atributo `oninput` do formulário.

Vamos começar com um formulário bem simples, pedindo informações sobre uma determinada compra:

Produto:

Preço: R\$

Quantidade:

Total: R\$ 0

O código básico para criar o formulário acima será:

```
1      <form action="cadastro.php" method="get">
2          <p><label for="iprod">Produto:</label>
3          <input type="text" name="prod" id="iprod"></p>
4          <p><label for="ipreco">Preço: R$</label>
5          <input type="number" name="preco" id="ipreco"></p>
6          <p><label for="iquant">Quantidade:</label>
7          <input type="number" name="quant" id="iquant"></p>
8          <p><label for="itot">Total: R$</label>
9          <output name="tot" id="itot">0</output></p>
10         <p><input type="submit" value="Enviar"></p>
11     </form>
```

A única novidade é a **linha 9**, onde temos um campo de saída que não pode ser editado pelo seu visitante e sequer será enviado pelo seu formulário. O seu objetivo aqui é manipular o seu conteúdo através de instruções simples.

Para calcular o Total apresentado ali embaixo, basta pegar o preço e multiplicar pela quantidade, não é? Pois então vamos fazer isso pegando o valor de cada elemento identificado pelo seu `id` transformado em número, o que ficaria:

```
itot.value = Number(ipreco.value) * Number(iquant.value)
```

A forma de "ler" a linha acima é:

*"O valor dentro do total (**itot**) recebe (=) o valor do preço (**ipreco**) convertido para número multiplicado (*) pelo valor da quantidade (**iquant**) convertido para número."*

Vamos pegar essa instrução JavaScript que construímos e colocar na **linha 1** do HTML escrito acima, dentro do atributo `oninput`:

```
<form action="cadastro.php" method="get" oninput="itot.value =
Number(ipreco.value) * Number(iquant.value)">
```


Não se esqueça de conferir se as letras maiúsculas e minúsculas estão sendo digitadas corretamente, pois isso faz total diferença para o JavaScript.

Agora faça um teste e veja se a sua operação está funcionando corretamente!

Produto:

Preço: R\$

Quantidade:

Total: R\$ 540

Viu como o total é calculado automaticamente, assim que digitamos preço e quantidade?



CUIDADO! Se não funcionar, provavelmente é o seu código JavaScript digitado errado, verifique se digitou todas as letras maiúsculas e minúsculas exatamente como colocamos aqui no material.



AS SAÍDAS NÃO SÃO ENVIADAS! O valor gerado dentro de um elemento `<output>` não será enviado quando pressionarmos o botão de submissão. Ele serve apenas para mostrar saídas ao seu visitante. Caso queira enviar os dados, pesquise mais sobre como funcionam os elementos do tipo `hidden`.

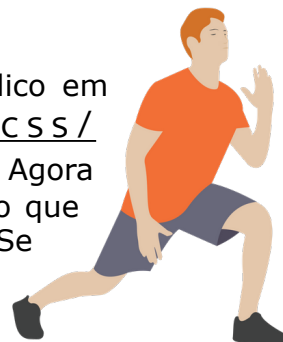


QUER APRENDER MAIS JAVASCRIPT? Se quiser se aprofundar um pouco mais no aprendizado de JavaScript, recomendo um curso completo com os primeiros passos na linguagem que eu criei e está completamente grátis:

Curso em Vídeo: https://www.youtube.com/playlist?list=PLHz_AreHm4dlsK3Nr9GVvXCbpQyHQ11o1

Hora de exercitar

Chegou a hora de acessar o endereço do nosso repositório público em <https://gustavoguanabara.github.io/html-css/desafios/> e executar os **exercícios ???** no seu computador. Agora tente atingir esse mesmo resultado em casa, sem copiar o código que eu criei. Nesse momento, a prática é algo que você mais precisa. Se por acaso ficar difícil, pode acessar o repositório público de HTML e CSS e dar uma olhada nos comandos, mas **EVITE COPIAR**.



Tenho desafios pra você!

Lá no repositório, além do material em PDF e dos códigos dos exercícios 100% disponíveis, também disponibilizamos alguns **desafios** que devem ser resolvidos. Esses desafios não incluem o código original e você deve tentar chegar à resposta sem copiar nenhum código.

Com todo o conteúdo que vimos até essa aula, você já pode resolver o **desafio ???**. Acesse o repositório público, abra a área do curso de HTML+CSS e clique no link de acesso aos desafios. Manda ver! Só não fica pedindo a resposta! Você consegue resolver isso sozinho(a)!



Repositório em: <https://gustavoguanabara.github.io>

Eu já falei sobre isso no YouTube?

Eu sei que às vezes as pessoas gostam mais de assistir vídeos do que ler livros, e é por isso que eu lanço há anos materiais no canal Curso em Vídeo no YouTube. O link que vou compartilhar contigo tem o conteúdo explicado como você leu aqui, só que de forma mais ilustrada. Reserve um tempo dos seus estudos para assistir esse vídeo todo.



Curso em Vídeo: https://www.youtube.com/playlist?list=PLHz_AreHm4dlAnJ_jJtV29RFxnPHDuk9o